Create a Spring Web Project using Maven

src > main > java > LoggingExample.java > ...

```java
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;

public class LoggingExample {
    private static final Logger logger = LoggerFactory.getLogger(LoggingExample.class);

    public static void main(String[] args) {
        logger.error("This is an error message");
        logger.warn("This is a warning message");
    }
}
```

pom.xml > { } Grammars > https://maven.apache.org/xsd/maven-4.0.0.xsd > Cache > file:///C:/Users/prane/.lemminx/cache/https/maven.apache.org/xsd/maven-4.0.0.xsd

```xml
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0" xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
    xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 https://maven.apache.org/xsd/maven-4.0.0.xsd">
    <modelVersion>4.0.0</modelVersion>
    <parent>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-parent</artifactId>
        <version>4.1.0</version>
        <relativePath/> <!-- lookup parent from repository -->
    </parent>
    <groupId>com.cognizant</groupId>
    <artifactId>spring-learn</artifactId>
    <version>0.0.1-SNAPSHOT</version>
    <name/>
    <description/>
    <url/>
    <licenses>
        <license/>
    </licenses>
    <developers>
        <developer/>
    </developers>
    <scm>
        <connection/>
        <developerConnection/>
        <tag/>
        <url/>
    </scm>
    <properties>
        <java.version>17</java.version>
    </properties>
    <dependencies> Add Spring Boot Starters...
        <dependency>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-starter-webmvc</artifactId>
        </dependency>

        <dependency>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-devtools</artifactId>
            <scope>runtime</scope>
            <optional>true</optional>
        </dependency>
        <dependency>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-starter-webmvc-test</artifactId>
            <scope>test</scope>
        </dependency>
    </dependencies>

    <build>
        <plugins>
            <plugin>
                <groupId>org.springframework.boot</groupId>
                <artifactId>spring-boot-maven-plugin</artifactId>
            </plugin>
        </plugins>
    </build>

</project>
```

```
: Graceful shutdown complete
[INFO] ------------------------------------------------------------------------
[INFO] BUILD SUCCESS
[INFO] ------------------------------------------------------------------------
[INFO] Total time:  01:28 min
[INFO] Finished at: 2026-07-08T18:52:35+05:30
[INFO] ------------------------------------------------------------------------
```

Spring Core – Load Country from Spring Configuration XML

src > main > java > com > cognizant > springlearn > J Country.java > { } com.cognizant.springlearn

```java
package com.cognizant.springlearn;

public class Country {

    private String code;
    private String name;

    /**
     * Default constructor
     */
    public Country() {
        System.out.println(x: "Country() - Constructor invoked");
    }

    /**
     * Parameterized constructor
     */
    public Country(String code, String name) {
        this.code = code;
        this.name = name;
        System.out.println(x: "Country(String code, String name) - Constructor invoked");
    }

    /**
     * Getter for code
     */
    public String getCode() {
        return code;
    }

    /**
     * Setter for code
     */
    public void setCode(String code) {
        System.out.println("setCode() - Method invoked with value: " + code);
        this.code = code;
    }

    /**
     * Getter for name
     */
    public String getName() {
        return name;
    }

    /**
     * Setter for name
     */
    public void setName(String name) {
        System.out.println("setName() - Method invoked with value: " + name);
        this.name = name;
    }

    @Override
    public String toString() {
        return "Country{" +
                "code='" + code + '\'' +
                ", name='" + name + '\'' +
                '}';
    }
}
```

src > main > java > com > cognizant > springlearn > J SpringLearnApplication.java > { } com.cognizant.springlearn

```java
package com.cognizant.springlearn;

import org.springframework.context.ApplicationContext;
import org.springframework.context.support.ClassPathXmlApplicationContext;
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;

/**
 * Spring Learn Application
 * This application demonstrates loading country configuration from Spring XML configuration file
 */
public class SpringLearnApplication {

    private static final Logger LOGGER = LoggerFactory.getLogger(SpringLearnApplication.class);

    /**
     * Main method - Entry point of the application
     */
    Run | Debug
    public static void main(String[] args) {
        LOGGER.debug("main() - Method invoked");

        // Display all countries
        LOGGER.debug("========== Displaying all countries ==========");
        displayCountry(countryBeanId: "countryUS");
        displayCountry(countryBeanId: "countryDE");
        displayCountry(countryBeanId: "countryIN");
        displayCountry(countryBeanId: "countryJP");

        LOGGER.debug("========== Application execution completed ==========");
    }

    /**
     * Method to read country bean from spring configuration file and display details
     * @param countryBeanId - The bean ID of the country to retrieve
     */
    private static void displayCountry(String countryBeanId) {
        LOGGER.debug("\ndisplayCountry() - Method invoked for bean: {}", countryBeanId);

        // Create ApplicationContext by loading the country.xml configuration file
        ApplicationContext context = new ClassPathXmlApplicationContext("country.xml");
        LOGGER.debug("ClassPathXmlApplicationContext created and country.xml loaded");

        // Get the country bean from the context
        Country country = context.getBean(countryBeanId, Country.class);
        LOGGER.debug("getBean() method invoked to retrieve bean with id: {}", countryBeanId);

        // Display the country details
        LOGGER.debug("Country : {}", country.toString());
        System.out.println(country.toString());
    }
}
```

src > main > resources > country.xml > { } Grammars > http://www.springframework.org/schema/bean

```xml
<?xml version="1.0" encoding="UTF-8" ?>
<beans xmlns="http://www.springframework.org/schema/beans"
    xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
    xsi:schemaLocation="http://www.springframework.org/schema/beans
    http://www.springframework.org/schema/beans/spring-beans.xsd">

    <!-- United States Country Bean -->
    <bean id="countryUS" class="com.cognizant.springlearn.Country">
        <property name="code" value="US" />
        <property name="name" value="United States" />
    </bean>

    <!-- Germany Country Bean -->
    <bean id="countryDE" class="com.cognizant.springlearn.Country">
        <property name="code" value="DE" />
        <property name="name" value="Germany" />
    </bean>

    <!-- India Country Bean -->
    <bean id="countryIN" class="com.cognizant.springlearn.Country">
        <property name="code" value="IN" />
        <property name="name" value="India" />
    </bean>

    <!-- Japan Country Bean -->
    <bean id="countryJP" class="com.cognizant.springlearn.Country">
        <property name="code" value="JP" />
        <property name="name" value="Japan" />
    </bean>

</beans>
```

src > main > resources > logback.xml > xml

```xml
<?xml version="1.0" encoding="UTF-8"?>
<configuration>
    <appender name="CONSOLE" class="ch.qos.logback.core.ConsoleAppender">
        <encoder>
            <pattern>%d{yyyy-MM-dd HH:mm:ss.SSS} [%thread] %-5level %logger{36} - %msg%n</pattern>
        </encoder>
    </appender>

    <root level="DEBUG">
        <appender-ref ref="CONSOLE" />
    </root>

    <logger name="com.cognizant.springlearn" level="DEBUG" />
</configuration>

```

```
PS D:\PROJECT\Spring> mvn exec:java
2026-07-09 15:21:55.223 [com.cognizant.springlearn.SpringLearnApplication.main()] DEBUG c.c.s.SpringLearnApplication - ========== Application execut
ion completed ==========
[INFO] ------------------------------------------------------------------------
[INFO] BUILD SUCCESS
[INFO] ------------------------------------------------------------------------
[INFO] Total time:  2.771 s
[INFO] Finished at: 2026-07-09T15:21:55+05:30
[INFO] ------------------------------------------------------------------------
```

Hello World RESTful Web Service

src > main > java > com > cognizant > springlearn > controller > HelloController.java > Language Support for Java(TM) by Red Hat > { } com.cognizant.s

```java
package com.cognizant.springlearn.controller;

import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.RestController;

@RestController
public class HelloController {
    private static final Logger logger = LoggerFactory.getLogger(HelloController.class);

    @GetMapping("/hello")
    public String sayHello() {
        logger.info("Start sayHello()");
        logger.info("End sayHello()");
        return "Hello World!!";
    }
}
```

src > main > java > com > cognizant > springlearn > SpringLearnApplication.java > Language Support for Java(TM) by Red Hat > { } c

```java
package com.cognizant.springlearn;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;

@SpringBootApplication
public class SpringLearnApplication {
    Run | Debug
    public static void main(String[] args) {
        SpringApplication.run(SpringLearnApplication.class, args);
    }
}
```

```
Hello World!!
```

Key	Value
Content-Type	text/plain;charset=UTF-8
Content-Length	13
Date	Thu, 09 Jul 2026 10:06:11 GMT
Keep-Alive	timeout=60
Connection	keep-alive

REST - Country Web Service

src > test > java > com > cognizant > springlearn > CountryControllerIntegrationTest.java > { } com.cognizant.springlearn

```java
package com.cognizant.springlearn;

import org.junit.jupiter.api.Test;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.boot.test.autoconfigure.web.servlet.AutoConfigureMockMvc;
import org.springframework.boot.test.context.SpringBootTest;
import org.springframework.http.MediaType;
import org.springframework.test.web.servlet.MockMvc;

import static org.springframework.test.web.servlet.request.MockMvcRequestBuilders.get;
import static org.springframework.test.web.servlet.result.MockMvcResultMatchers.content;
import static org.springframework.test.web.servlet.result.MockMvcResultMatchers.jsonPath;
import static org.springframework.test.web.servlet.result.MockMvcResultMatchers.status;

@SpringBootTest
@AutoConfigureMockMvc
class CountryControllerIntegrationTest {

    @Autowired
    private MockMvc mockMvc;

    @Test
    void getCountryIndiaReturnsIndiaDetails() throws Exception {
        mockMvc.perform(get("/country"))
                .andExpect(status().isOk())
                .andExpect(content().contentTypeCompatibleWith(MediaType.APPLICATION_JSON))
                .andExpect(jsonPath("$.code").value("IN"))
                .andExpect(jsonPath("$.name").value("India"));
    }
}

```

src > main > java > com > cognizant > springlearn > CountryWebServiceApplication.java > Language Support for Java(TM) by Red Hat > { } com.cognizant.springlearn

```java
package com.cognizant.springlearn;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;
import org.springframework.context.annotation.ImportResource;

@SpringBootApplication
@ImportResource("classpath:country-beans.xml")
public class CountryWebServiceApplication {
    Run | Debug
    public static void main(String[] args) {
        SpringApplication.run(CountryWebServiceApplication.class, args);
    }
}

```

```json
{
    "code": "IN",
    "name": "India"
}
```

Key	Value
Content-Type	application/json
Transfer-Encoding	chunked
Date	Thu, 09 Jul 2026 10:46:03 GMT
Keep-Alive	timeout=60
Connection	keep-alive

REST - Get country based on country code

src > main > java > com > cognizant > springlearn > controller > CountryController.java > Language Support for Java(TM) by Red Hat > { } com.cogni

```java
package com.cognizant.springlearn.controller;

import com.cognizant.springlearn.model.Country;
import com.cognizant.springlearn.service.CountryService;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.PathVariable;
import org.springframework.web.bind.annotation.RestController;

@RestController
public class CountryController {

    private final CountryService countryService;

    public CountryController(CountryService countryService) {
        this.countryService = countryService;
    }

    @GetMapping("/countries/{code}")
    public Country getCountry(@PathVariable String code) {
        return countryService.getCountry(code);
    }
}

```

src > main > java > com > cognizant > springlearn > model > Country.java > Country

```java
package com.cognizant.springlearn.model;

public class Country {
    private String code;
    private String name;

    public Country() {
    }

    public Country(String code, String name) {
        this.code = code;
        this.name = name;
    }

    public String getCode() {
        return code;
    }

    public void setCode(String code) {
        this.code = code;
    }

    public String getName() {
        return name;
    }

    public void setName(String name) {
        this.name = name;
    }
}

```

src > main > java > com > cognizant > springlearn > CountryCodeWebServiceApplication.java > Language Support for Java(TM) by Red H

```java
package com.cognizant.springlearn;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;

@SpringBootApplication
public class CountryCodeWebServiceApplication {
    Run | Debug
    public static void main(String[] args) {
        SpringApplication.run(CountryCodeWebServiceApplication.class, args);
    }
}

```


```
{
    "code": "IN",
    "name": "India"
}
```


Key	Value
Content-Type	application/json
Transfer-Encoding	chunked
Date	Thu, 09 Jul 2026 11:02:08 GMT
Keep-Alive	timeout=60
Connection	keep-alive

Create authentication service that returns JWT

src > main > java > com > example > jwtservice > JwtServiceApplication.java > Language Support for Java(TM) by Red Hat > { } com.example.jwtservice

```java
package com.example.jwtservice;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;

@SpringBootApplication
public class JwtServiceApplication {
    Run | Debug
    public static void main(String[] args) {
        SpringApplication.run(JwtServiceApplication.class, args);
    }
}
```

src > test > java > com > example > jwtservice > AuthenticationControllerTest.java > { } com.example.jwtservice

```java
package com.example.jwtservice;

import org.junit.jupiter.api.Test;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.boot.test.autoconfigure.web.servlet.AutoConfigureMockMvc;
import org.springframework.boot.test.context.SpringBootTest;
import org.springframework.http.MediaType;
import org.springframework.test.web.servlet.MockMvc;

import static org.springframework.test.web.servlet.request.MockMvcRequestBuilders.get;
import static org.springframework.test.web.servlet.result.MockMvcResultMatchers.jsonPath;
import static org.springframework.test.web.servlet.result.MockMvcResultMatchers.status;

@SpringBootTest
@AutoConfigureMockMvc
class AuthenticationControllerTest {

    @Autowired
    private MockMvc mockMvc;

    @Test
    void shouldReturnTokenForBasicAuthCredentials() throws Exception {
        mockMvc.perform(get("/authenticate")
                        .header("Authorization", "Basic dXNlcjpwd2Q=")
                        .accept(MediaType.APPLICATION_JSON))
                .andExpect(status().isOk())
                .andExpect(jsonPath("$.token").isString());
    }
}
```

```
{
    "token": "eyJhbGciOiJIUzI1NiJ9.eyJzdWIiOiJ1c2VyIiwiaWF0IjoxNzgzNTk1OTk4LCJleHAiOjE3ODM1OTk1OTh9.6lPuNFpO4Vbvt0oUuxr3ryPlIhWT1AfMNTSQmEBZTYs"
}
```